

Deepfake Detection: Real vs StyleGAN-Generated Faces

Transfer learning with EfficientNet-B0 and ResNet-18, explained with Grad-CAM,
deployed as a Streamlit demo

Author	Mohd Shaan
Submitted to	Dr. Prakash
Course	Machine Learning — Semester 5
Date	October 2026
Code & notebooks	github.com/Mohd2Shaan/Deepfake-Detection-Project

99.84%

test accuracy
(EfficientNet-B0)

1.0000

ROC-AUC

33 / 20,000

test images misclassified

140,000

face images used

Abstract

Modern generative models can create photorealistic human faces of people who do not exist. This project builds a binary image classifier that decides whether a face image is a **real photograph** or a **StyleGAN-generated fake**. Using the public *140k Real and Fake Faces* dataset (70,000 real FFHQ photographs and 70,000 StyleGAN images), we fine-tune two ImageNet-pretrained convolutional neural networks — **EfficientNet-B0** and **ResNet-18** — with a two-phase transfer-learning schedule (frozen warm-up, then partial fine-tuning). An exploratory analysis confirmed a clean, balanced dataset with no trivial shortcuts. A first EfficientNet run that fine-tuned only 28% of its weights under-fitted (93.71% validation accuracy); unfreezing four blocks (92% of weights) raised it to 99.88%. On the held-out test set of 20,000 images the final EfficientNet-B0 reaches **99.84% accuracy, 99.84% F1 and a ROC-AUC of 1.0000** (33 errors), against 98.98% for ResNet-18 (205 errors) — with about three times fewer parameters. Grad-CAM heatmaps show that the model detects fakes by inspecting the inner face (eyes, nose bridge, mouth) and recognises real photos from hair, background and surrounding objects. A Streamlit application exposes the model with a confidence score and heatmap. Tests with images generated by ChatGPT were all classified as real with $\approx 100\%$ confidence, demonstrating the key limitation: a detector trained on one generator does not generalise to others. We discuss this limitation and outline concrete extensions.

Contents

1	Introduction and Problem Statement
2	Background and Literature Review
3	Dataset and Exploratory Data Analysis
4	Methodology
4.1	Pipeline · 4.2 Preprocessing and augmentation · 4.3 Model architectures · 4.4 Two-phase transfer learning · 4.5 Loss, optimiser, scheduler and checkpointing · 4.6 Evaluation protocol · 4.7 Grad-CAM · 4.8 Implementation
5	Experiments and Results
5.1	Setup · 5.2 Training runs · 5.3 Ablation: how much to fine-tune · 5.4 Test-set results · 5.5 Error analysis · 5.6 Grad-CAM findings · 5.7 Demo application and out-of-distribution test
6	Discussion
7	Limitations
8	Future Work and Extensions
9	Conclusion
	References
	Appendix A — Reproducing the results · Appendix B — Viva questions

1 Introduction and Problem Statement

1.1 Deepfakes

“Deepfake” (*deep learning* + *fake*) refers to images, video or audio generated or manipulated by deep neural networks so that they appear authentic. Two broad families exist: **face manipulation**, where a real video is edited (face swap, lip-sync, expression transfer), and **entire face synthesis**, where a new face that never existed is produced from random noise by a generative model such as StyleGAN. Synthetic faces are used for fake social-media profiles, fraud, and disinformation, and humans cannot reliably tell them apart from photographs — our own EDA sample grids (Section 3) illustrate this. Automatic detection is therefore an important tool for platforms, journalists and forensic analysts.

1.2 Problem statement

Given a single RGB image of a human face, predict whether it is **real** (a photograph) or **fake** (generated by StyleGAN). This is a **binary image-classification** problem. The model outputs one number, the probability $P(fake)$; the image is labelled fake when $P(fake) \geq 0.5$.

1.3 Objectives

1. Build a clean, reproducible data pipeline and verify the data with exploratory analysis.
2. Fine-tune two ImageNet-pretrained CNNs (EfficientNet-B0 as primary model, ResNet-18 as comparison) and compare them.
3. Evaluate the better model rigorously on an untouched test set (accuracy, precision, recall, F1, ROC-AUC, confusion matrix) and analyse its mistakes.
4. Explain the model’s decisions with Grad-CAM heatmaps.
5. Deploy the model in a simple Streamlit application and test it on images from outside the dataset.

1.4 Scope

The project targets **entire-face synthesis by StyleGAN** on **still images**. Video deepfakes, face swaps and other generators are outside the training scope; how the model behaves on them is examined only qualitatively (Section 5.7) and discussed as a limitation.

2 Background and Literature Review

2.1 Generative Adversarial Networks and StyleGAN

A **GAN** (Goodfellow et al., 2014) trains two networks against each other: a **generator** G maps a random vector z to an image, and a **discriminator** D tries to distinguish generated images from real ones. G improves by fooling D ; at equilibrium the generated images follow the real-image distribution. **StyleGAN** (Karras, Laine & Aila, 2019) first maps z through an 8-layer *mapping network* into an intermediate “style” space w , then injects the style at every resolution of the synthesis network (coarse layers control pose and face shape, fine layers control skin and hair texture) and adds per-pixel noise for stochastic detail. The same paper introduced **FFHQ**, the 70,000-image high-quality face dataset whose photographs form the “real” half of our data.

Even high-quality GAN images keep traces of their generation process: repeated up-sampling leaves periodic patterns in the frequency spectrum (Durall et al., 2020; Frank et al., 2020), and StyleGAN images show characteristic artifacts such as “water-droplet” blobs, asymmetric accessories, malformed teeth and ears, and smeared hair/background boundaries. A CNN can learn such cues from data without being told about them explicitly.

2.2 Deepfake detection

- **FaceForensics++** (Rössler et al., 2019) introduced a large benchmark of manipulated face videos (DeepFakes, Face2Face, FaceSwap, NeuralTextures) and showed that a CNN (XceptionNet) trained on face crops clearly outperforms hand-crafted forensic features — but that heavy video compression strongly reduces accuracy.
- **Wang et al. (2020)**, “CNN-generated images are surprisingly easy to spot... for now”, trained a ResNet-50 on ProGAN images with blur/JPEG augmentation and found it generalised surprisingly well to other GANs, supporting the use of standard pretrained CNNs for this task.
- **Frequency-domain methods** (Durall et al., 2020; Frank et al., 2020) showed that GAN up-sampling produces spectral artifacts that make fakes separable even with simple classifiers.
- **Surveys** (Tolosana et al., 2020) categorise manipulations into entire-face synthesis, identity swap, attribute manipulation and expression swap, and note that synthesis detection on a known generator is close to solved while **generalisation to unseen generators** remains open.
- **Generalisation:** Ojha et al. (2023) showed that detectors trained on one generator fail on others and proposed using frozen CLIP features with a linear probe; Corvi et al. (2023) showed that GAN-trained detectors miss diffusion-model images. Both directly motivate our limitations and future work.

2.3 Backbone architectures

ResNet (He et al., 2016) introduced residual connections $y = F(x) + x$, which let gradients flow through very deep networks; ResNet-18 is a compact, well-understood baseline. **EfficientNet** (Tan & Le, 2019) scales depth, width and input resolution together with one compound coefficient and is built from MBConv blocks (inverted bottlenecks with depthwise convolutions and squeeze-and-excitation); EfficientNet-B0 matches or exceeds ResNet-50 accuracy on ImageNet with about 5.3 M parameters.

2.4 Explainability with Grad-CAM

Grad-CAM (Selvaraju et al., 2017) produces a class-specific heatmap from any CNN by weighting the feature maps of a convolutional layer by the average gradient of the class score. It requires no architectural change and shows *where* in the image the evidence for a decision lies.

2.5 Positioning of this project

We follow the established recipe — fine-tune an ImageNet CNN on aligned face crops — and add (i) a controlled comparison of two architectures and of how much of the network is fine-tuned, (ii) systematic error analysis, (iii) Grad-CAM interpretation of correct and incorrect predictions, and (iv) a deployed demo with an out-of-distribution check that makes the generalisation problem visible.

3 Dataset and Exploratory Data Analysis

3.1 The 140k Real and Fake Faces dataset

What	140,000 face images at 256×256 px: 70,000 real photographs from FFHQ (Flickr-Faces-HQ) and 70,000 fakes generated by StyleGAN, published on Kaggle (author: xhlulu).
Why	Large, perfectly balanced, already face-aligned, and pre-split into train/validation/test — ideal for a binary classifier and small enough to train on a free GPU.
How	Used directly on Kaggle (attached as a notebook input — no download) with the provided splits. A helper script (<code>scripts/download_dataset.py</code>) arranges it into <code>data/{train,valid,test}/{real,fake}</code> for local or Colab use.

Split	Real	Fake	Total	Purpose
train	50,000	50,000	100,000	learning the weights
valid	10,000	10,000	20,000	choosing the checkpoint, LR scheduling
test	10,000	10,000	20,000	final, one-time evaluation only

Avoiding data leakage. The provided splits are used exactly as given and never reshuffled. Some real/fake images in this dataset are related, and re-splitting could place near-duplicates on both sides of the train/test boundary, inflating the scores. The test set was touched only once, after all modelling decisions were final.

3.2 Exploratory data analysis

What	A full scan of all 140,000 images: counts per class and split, image size and colour mode, file integrity, exact duplicates, JPEG file-size distributions, sample grids, the mean face and mean Fourier spectrum per class, and a visual check of transformed training batches.
Why	“Garbage in, garbage out”: corrupted files, imbalance, duplicates across splits or trivial shortcuts (e.g. one class being more compressed) would make any accuracy number meaningless.
How	<code>src/eda.py</code> opens every file (<code>PIL.Image.verify()</code> for integrity), hashes it with MD5 for duplicate detection, and produces the plots; run in <code>notebooks/01_eda.ipynb</code> on Kaggle (≈17 min).

Check	Result
Class balance	exactly 50 / 50 in every split
Image size and mode	all 140,000 images are 256 × 256, RGB
Corrupted files	0
Exact duplicates (MD5), within and across splits	0
Mean JPEG size	real 28.26 KB (σ 3.88) · fake 27.76 KB (σ 3.86)



Figure 1a. Random real (FFHQ) faces.

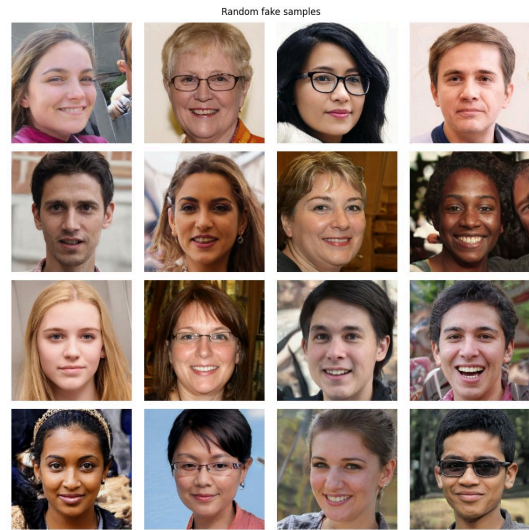


Figure 1b. Random StyleGAN faces — visually indistinguishable.

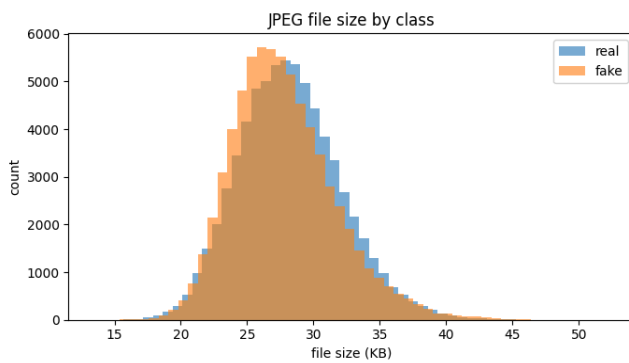


Figure 2. JPEG file size per class — almost complete overlap.

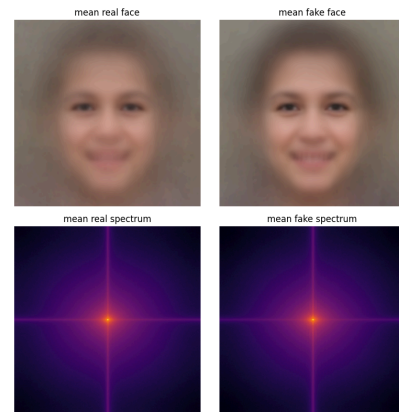


Figure 3. Mean face and mean log-Fourier spectrum per class.

Findings

1. **Clean and balanced** — no cleaning step needed; with 50/50 classes, accuracy is a fair headline metric and 0.5 is a natural threshold.
2. **No compression shortcut** — fakes are only ≈ 0.5 KB smaller on average (≈ 0.13 standard deviations), consistent with slightly smoother GAN textures, but far too weak to separate the classes.
3. **Humans cannot tell them apart** — both sample grids look like ordinary portraits; some fakes have odd backgrounds (warped objects, half-formed second faces).
4. **Mean faces** — both classes are face-centred and aligned; the mean fake is sharper, i.e. StyleGAN outputs are more uniform in pose.
5. **Mean spectra are nearly identical** — no obvious up-sampling peaks in the averaged spectrum at 256 px (the bright cross is a border effect of the FFT). The signal is subtle and local, which motivates a CNN instead of a global frequency rule.

Fourier recap. The 2-D discrete Fourier transform writes an image as a sum of sinusoids, $F(u,v) = \sum_x \sum_y f(x,y) \cdot e^{-2\pi i(ux/M + vy/N)}$; after centring, low frequencies (smooth shading) are in the middle and high frequencies (edges, fine texture, noise) towards the borders — where GAN up-sampling artifacts would appear.

4 Methodology

4.1 Overall pipeline

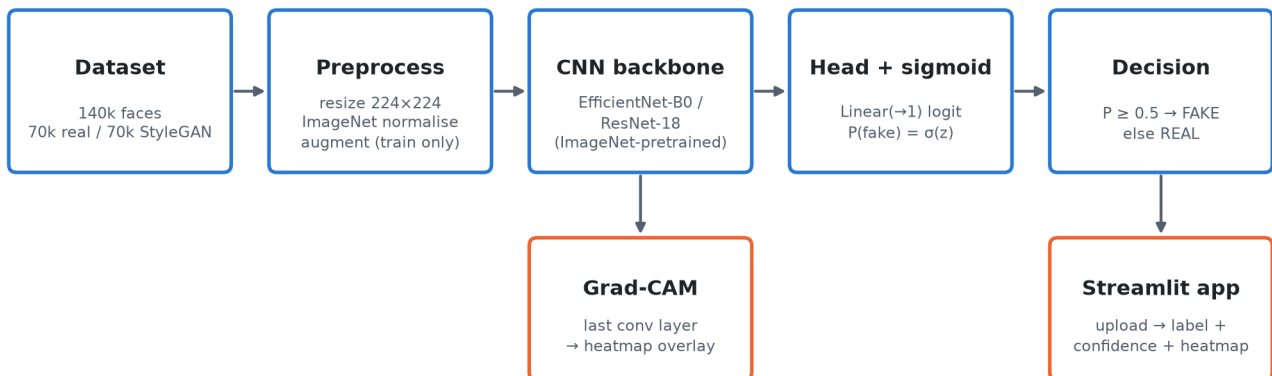


Figure 4. End-to-end pipeline. Blue: classification path; orange: explanation and deployment.

The project was built **module by module**, each with its own code file, notebook and documentation: data pipeline (`src/dataset.py`) → EDA (`src/eda.py`) → models (`src/model.py`) → training (`src/train.py`) → evaluation (`src/evaluate.py`) → Grad-CAM (`src/gradcam.py`) → app (`app.py`). All shared paths and hyper-parameters live in `src/config.py`.

4.2 Preprocessing and data augmentation

What	Resize every image to 224×224, convert to a tensor in [0, 1], normalise each channel with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). For training images only, add random horizontal flip ($p = 0.5$), random rotation ($\pm 10^\circ$) and colour jitter (brightness and contrast $\pm 20\%$).
Why	224×224 is the resolution the backbones were pretrained at (and 23% cheaper than 256^2). ImageNet normalisation gives inputs the same distribution the pretrained filters expect, and zero-centred inputs keep gradients well scaled. Augmentation is a regulariser: the network sees a slightly different version of each image every epoch and cannot memorise pixels. Transforms are mild and label-preserving — a mirrored or slightly tilted face is still real/fake — and deliberately avoid blur or JPEG re-compression, which could erase the high-frequency artifacts we want the model to learn. Validation/test images are not augmented so that evaluation is deterministic and reflects the true data distribution.
How	<code>torchvision.transforms</code> pipelines built by <code>get_transforms(train)</code> ; a custom <code>DeepfakeDataset</code> lists (path, label) pairs and opens images lazily; <code>DataLoader</code> batches 32 images, shuffles only the training set, and loads data in parallel workers.

Labels: **real** = 0, **fake** = 1, stored as floats because the loss compares a real-valued logit with a float target. The sanity-check batch below confirmed correct labels and visually reasonable augmentation; the black corners introduced by rotation appear in both classes equally and therefore cannot leak the label.



Figure 5. A training batch after augmentation (flip, rotation, colour jitter) with its labels.

4.3 Model architectures

What

Two ImageNet-pretrained CNNs with their 1000-class head replaced by a single-output linear layer: EfficientNet-B0 (`classifier[1] = Linear(1280, 1)` , keeping its dropout 0.2) and ResNet-18 (`fc = Linear(512, 1)`).

Why

Transfer learning: early layers of an ImageNet network already detect edges, textures and colour blobs that are useful for any image task, so we only need to adapt the later layers — much faster and more accurate than training from scratch on a free GPU. **EfficientNet-B0** offers the best accuracy per parameter; **ResNet-18** is a simple, well-known residual baseline that shows whether the result depends on the architecture. **One output** instead of two: for a binary task a single logit z with $P(fake) = \sigma(z) = 1/(1+e^{-z})$ is mathematically equivalent to a 2-way softmax, with fewer parameters.

How

`src/model.py` builds both with the torchvision weights API (`EfficientNet_B0_Weights.IMAGENET1K_V1` , `ResNet18_Weights.IMAGENET1K_V1`) and stores the model name inside every checkpoint so evaluation, Grad-CAM and the app can rebuild the right architecture automatically.

EfficientNet-B0

Compound scaling sets depth $d = \alpha^\varphi$, width $w = \beta^\varphi$ and resolution $r = \gamma^\varphi$ with $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$; B0 ($\varphi = 0$) is the baseline found by architecture search. Its building block, **MBConv**, expands channels with a 1×1 convolution, applies a cheap **depthwise** $k \times k$ convolution (one filter per channel), re-weights channels with **squeeze-and-excitation** (global pooling $\rightarrow$ small MLP $\rightarrow$ sigmoid gates), projects back with a 1×1 convolution and adds a skip connection; the activation is SiLU, $x \cdot \sigma(x)$. In torchvision, `features` has 9 children: a stem, seven MBConv stages ($16 \rightarrow 320$ channels) and a final 1×1 convolution to 1280 channels, followed by global average pooling.

ResNet-18

A 7×7 stem convolution and max-pool, then four stages (layer1 – layer4, $64 \rightarrow 512$ channels) of two **BasicBlocks** each (3×3 conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow 3 \times 3$ conv $\rightarrow$ BN, plus the identity shortcut), global average pooling and the linear head. The shortcut means each block only learns a residual, which keeps deep networks trainable.

Model	Parameters	GFLOPs / image	Grad-CAM target layer	Feature map
EfficientNet-B0	4,008,829	≈ 0.39	features[-1]	$1280 \times 7 \times 7$
ResNet-18	11,177,025	≈ 1.8	layer4[-1]	$512 \times 7 \times 7$

4.4 Two-phase transfer learning

Epoch schedule (10 epochs, best checkpoint by validation loss)



Figure 6. Training schedule: 3 frozen warm-up epochs followed by 7 fine-tuning epochs.

What	Phase 1 (epochs 1–3): all backbone weights frozen; only the new head trains (lr $1e-4$). Phase 2 (epochs 4–10): the head plus the last n backbone blocks are unfrozen and trained with a new optimiser.
Why	The new head starts random. Training everything at once would send large, random gradients into the pretrained layers and damage their features. Warming the head up first, then fine-tuning only the most task-specific (last) blocks with a small learning rate, adapts the network while preserving generic early features (“catastrophic forgetting” is avoided). Fine-tuning fewer layers also reduces over-fitting risk and epoch time.
How	Freezing sets <code>requires_grad = False</code> ; the optimiser only receives trainable parameters. Frozen BatchNorm layers are kept in <code>eval()</code> mode so their running statistics stay at their ImageNet values (otherwise <code>model.train()</code> would silently keep updating them). “Last two blocks” = <code>features[7] + features[8]</code> for EfficientNet and <code>layer3 + layer4</code> for ResNet; the number of blocks is a command-line option (<code>--unfreeze-blocks</code>).

Configuration	Phase-1 trainable	Phase-2 trainable	Share of all weights
EfficientNet-B0, last 2 blocks (run 02)	1,281	1,130,673	28%
EfficientNet-B0, last 4 blocks (run 02b, final)	1,281	3,700,169	92%
ResNet-18, layer3 + layer4 (run 02)	513	10,493,953	94%

4.5 Loss, optimiser, scheduler and checkpointing

Binary cross-entropy with logits

$$BCE(y, p) = -[y \cdot \log p + (1 - y) \cdot \log(1 - p)], \quad p = \sigma(z)$$

The loss is small when the predicted probability is close to the true label and grows without bound for confident mistakes. `BCEWithLogitsLoss` takes the raw logit and fuses the sigmoid with the log in a numerically stable form, which is why the network has no sigmoid layer; the sigmoid is applied only at prediction time.

Adam optimiser

$$m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2, \quad w \leftarrow w - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

Adam keeps a running average of the gradient (momentum, smoothing noisy mini-batch gradients) and of its square (giving every parameter its own adaptive step size). It works well for fine-tuning with little tuning.

ReduceLROnPlateau and checkpointing

If the validation loss does not improve for one epoch, the learning rate is halved, letting the model settle into a minimum. After every epoch the model is evaluated on the validation set; whenever validation loss reaches a new minimum the weights are saved as `<model>_best.pth` — a form of **early stopping** that keeps the best-generalising weights even if later epochs get worse. We select by validation *loss* rather than accuracy because, near 99% accuracy, loss still reflects how confident and well-calibrated the predictions are. A full training state (`_last.pth`: weights, optimiser, scheduler, history) is saved every epoch so an interrupted cloud session can `--resume`.

Other details

Mixed-precision training (`torch.autocast` + `GradScaler`) on the GPU roughly halves memory and speeds up training; a fixed seed (42) makes runs reproducible — the frozen epochs of two separate EfficientNet runs matched to four decimals.

4.6 Evaluation protocol and metrics

Treating **fake as the positive class**, with TP, TN, FP, FN from the confusion matrix:

Metric	Formula	Question it answers
Accuracy	$(TP + TN) / N$	What fraction of all images is correct?
Precision	$TP / (TP + FP)$	When the model says “fake”, how often is it right?
Recall (TPR)	$TP / (TP + FN)$	Of all fakes, how many are caught?
F1	$2PR / (P + R)$	Balance of precision and recall
ROC-AUC	area under TPR vs FPR over all thresholds	Probability a random fake is scored above a random real image (threshold-independent)

The test set is used once, after training. In addition we list the 25 **most confident mistakes** (confidence = $P(fake)$ if predicted fake, else $1 - P(fake)$) — the cases where the model was sure and wrong are the most informative for understanding its limits.

4.7 Grad-CAM

What	A heatmap showing which image regions pushed the model towards its decision.
Why	A detector should rely on meaningful evidence (artifacts on the face, hair, background) rather than a shortcut (a watermark, border, compression level). Heatmaps build trust, help debug errors and explain results.
How	For feature maps A^k of the last convolutional layer and score y : weights $\alpha_k = (1/Z) \sum_{ij} \partial y / \partial A^k_{ij}$ (global-average-pooled gradients), map $L = ReLU(\sum_k \alpha_k A^k)$, upsampled from 7×7 to 224×224 and overlaid. For our single-logit model, $y = z$ explains “fake” and $y = -z$ explains “real”; we always explain the predicted class. Implemented with <code>pytorch-grad-cam</code> and, for understanding, re-implemented by hand with forward/backward hooks (<code>manual_gradcam</code>) — the two agree with correlation 0.99999999.

The last convolutional block is chosen because it holds the most class-specific features while still preserving spatial layout (the following global pooling discards location). Its 7×7 resolution makes the heatmaps coarse.

4.8 Implementation and engineering

Component	Tool / file
Language and framework	Python 3, PyTorch, torchvision
Data, metrics, plots	Pillow, NumPy, pandas, scikit-learn, matplotlib
Explainability	<code>pytorch-grad-cam</code> (<code>src/gradcam.py</code>)
Demo	Streamlit (<code>app.py</code>)
Compute	Kaggle Notebooks, NVIDIA Tesla T4 GPU (free tier); laptop CPU for the app
Version control	Git/GitHub, one commit per module, CHANGELOG, a theory document per module in <code>docs/</code>

Cloud workflow. Each notebook clones the repository, finds the attached Kaggle dataset automatically (`config.find_kaggle_dataset()` searches `/kaggle/input` at any depth, after Kaggle changed its mount path), and for later stages imports the trained checkpoints from the attached output of the training notebook. Long runs were executed as background “Save & Run All” commits; a temporary Kaggle dataset-mount failure was worked around by running evaluation interactively.

5 Experiments and Results

5.1 Setup

Setting	Value
Hardware	Kaggle, 1 × NVIDIA Tesla T4 (16 GB), 4 CPU cores
Input / batch size	224 × 224 RGB / 32
Schedule	3 frozen + 7 fine-tune epochs, Adam, ReduceLROnPlateau (factor 0.5, patience 1)
Time per epoch	≈ 7 min for both models (data loading bound); first epoch slower (cold disk cache)
Total GPU time	run 02 (both models) ≈ 3.0 h · run 02b (EfficientNet, 4 blocks) ≈ 1.5 h

5.2 Training runs

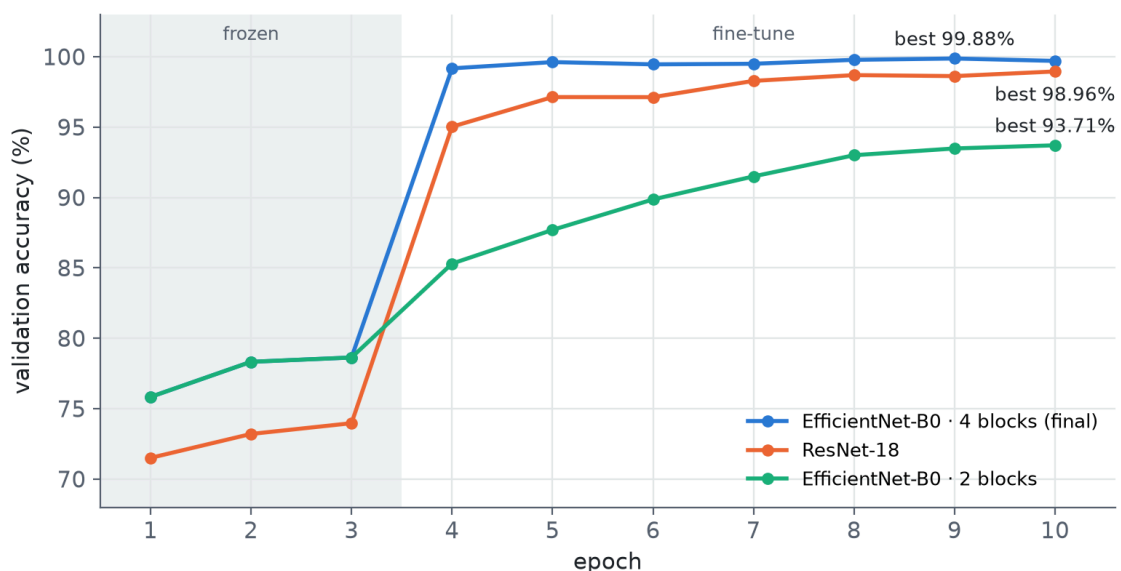


Figure 7. Validation accuracy per epoch for the three training runs (grey: frozen phase).

Run	Fine-tuned	Fine-tune lr	Best epoch	Val loss	Val accuracy	Train accuracy (ep. 10)
02 · EfficientNet-B0	last 2 blocks (28%)	1e-5	10	0.1581	93.71%	91.37%
02 · ResNet-18	layer3 + layer4 (94%)	1e-5	10	0.0290	98.96%	98.42%
02b · EfficientNet-B0	last 4 blocks (92%)	1e-4	9	0.0034	99.88%	99.72%

- **The frozen phase plateaus below 80%** for all runs: ImageNet features alone are only moderately useful for spotting GAN artifacts — the features themselves must adapt.
- **Fine-tuning causes an immediate jump** at epoch 4 (EfficientNet-4 blocks 78.6% → 99.2%, ResNet 74.0% → 95.0%).

- **No over-fitting:** in every run training and validation accuracy stay close, and validation loss keeps falling or stabilises. In run 02b the scheduler halved the learning rate after a plateau at epochs 6–7; epoch 10 was slightly worse than epoch 9, so the epoch-9 weights were kept.

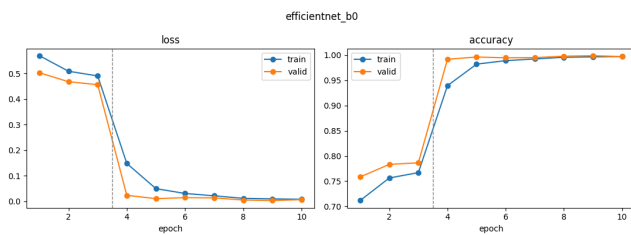


Figure 8a. EfficientNet-B0, 4 blocks (final).

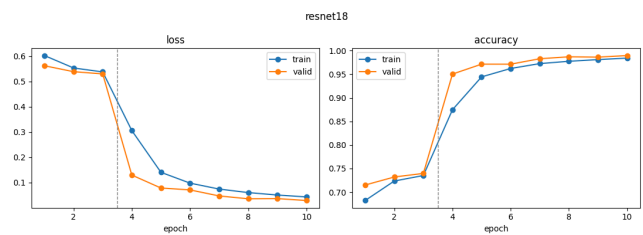


Figure 8b. ResNet-18.

5.3 Ablation: how much of the network to fine-tune

In run 02 EfficientNet-B0 clearly under-fitted: its validation loss was still falling at epoch 10 and its training accuracy (91.4%) was even below validation accuracy (93.7%) — augmentation and dropout make training images harder than validation images, and the model had not finished learning. The cause was the configuration, not the architecture: EfficientNet’s last two blocks hold only **28%** of its weights, whereas ResNet’s layer3 + layer4 hold **94%**, so with the same cautious learning rate ResNet adapted far more of its network. Unfreezing four blocks (92%) and using lr 1e-4 (run 02b) — both existing command-line options, no code change — raised EfficientNet to **99.88%**.

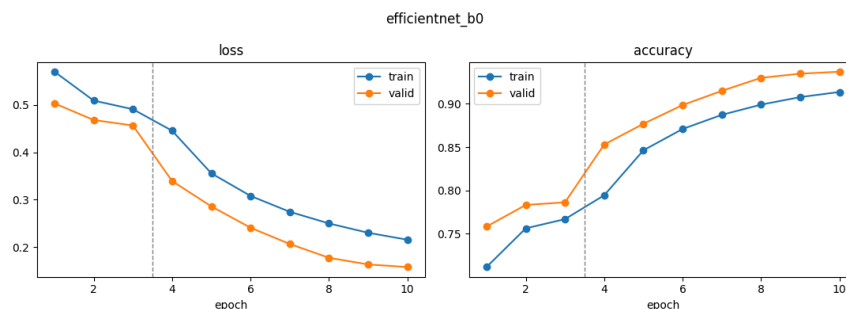


Figure 9. Run 02, EfficientNet-B0 with 2 blocks: slow, still-improving curves — under-fitting.

Lesson. How much of a pretrained network is fine-tuned (and how fast) mattered more than the choice between the two architectures. With a comparable share of trainable weights, EfficientNet-B0 beats ResNet-18 with about 3× fewer parameters.

5.4 Test-set results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Errors / 20,000
EfficientNet-B0 (final)	99.84%	99.79%	99.88%	99.84%	1.0000	33
ResNet-18	98.98%	98.58%	99.38%	98.98%	0.9994	205

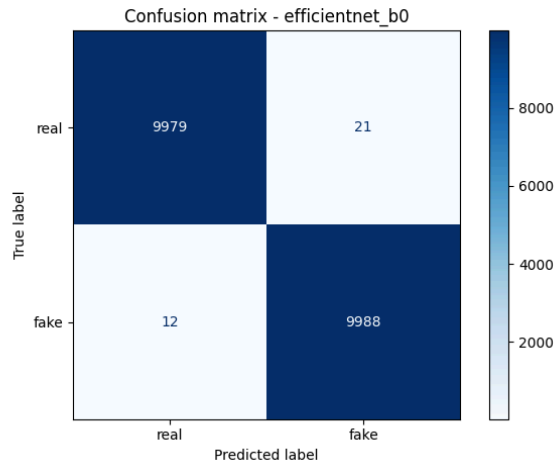


Figure 10a. Confusion matrix, EfficientNet-B0.

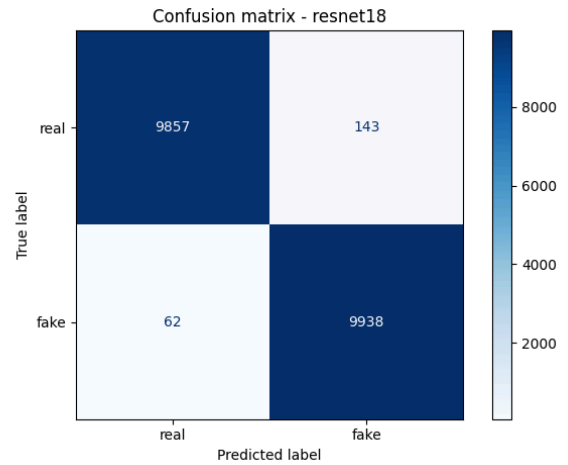


Figure 10b. Confusion matrix, ResNet-18.

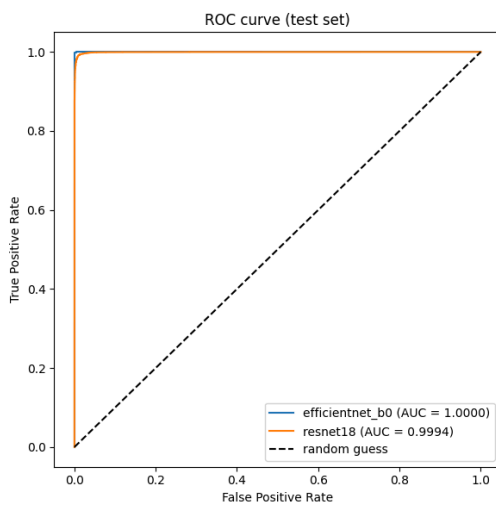


Figure 11. ROC curves on the test set.

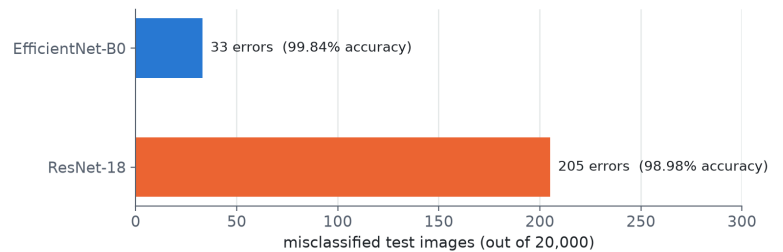


Figure 12. Misclassified test images per model.

- **Test $\approx$ validation** (EfficientNet 99.84% vs 99.88%; ResNet 98.98% vs 98.96%): the models generalise to unseen images of the same distribution and the checkpoint selection did not over-fit the validation set.
- **EfficientNet-B0 makes about 6 $\times$ fewer errors** than ResNet-18 and is selected as the final model.
- **ROC-AUC $\approx$ 1.000**: almost every fake receives a higher $P(fake)$ than almost every real image, so the result does not depend on the 0.5 threshold.
- Both models produce **more false positives than false negatives** (EfficientNet: 21 real photos flagged as fake vs 12 fakes missed; ResNet: 143 vs 62).

5.5 Error analysis

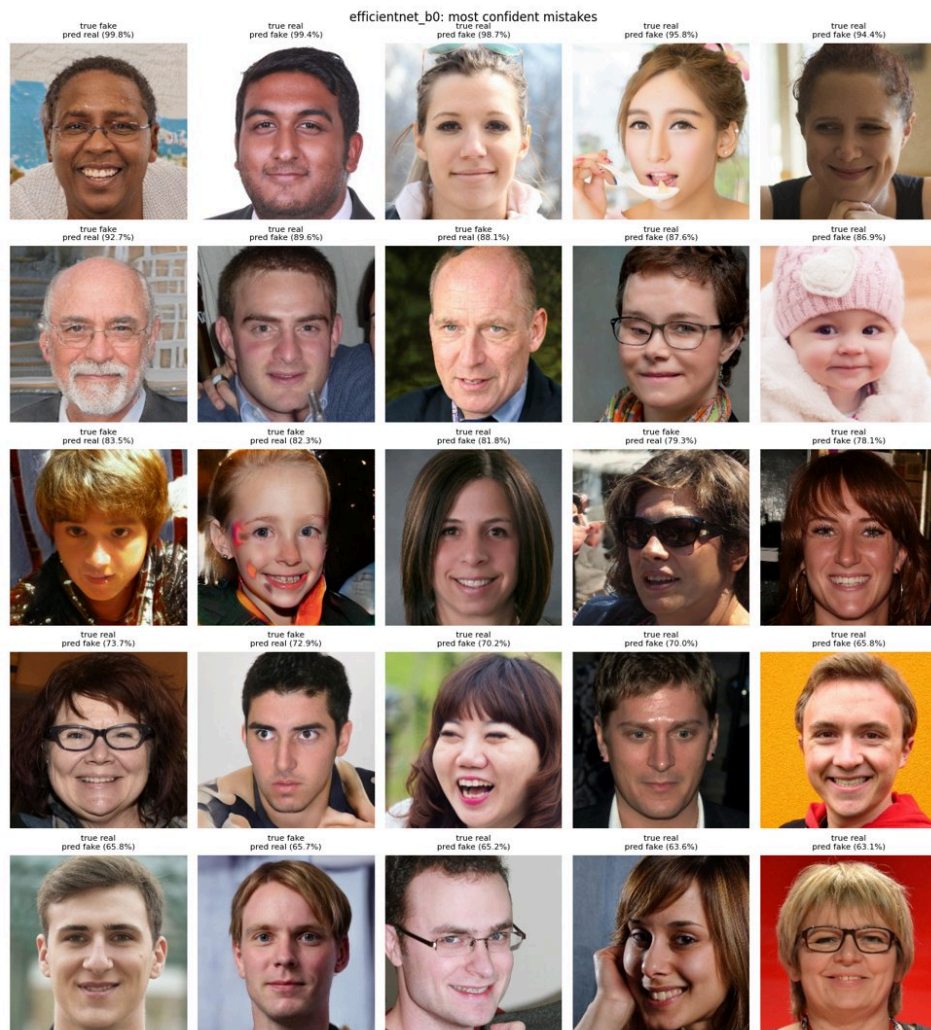


Figure 13. The 25 most confident mistakes of EfficientNet-B0 (title: true label, prediction, confidence).

- **Real → fake (false positives):** studio-like portraits with plain backgrounds, smooth or retouched skin, heavy make-up, strong flash or very symmetric frontal poses — real photographs that look as “clean” as StyleGAN output.
- **Fake → real (false negatives):** fakes with busy, realistic backgrounds or clothing, unusual lighting, or accessories (sunglasses, face paint) where StyleGAN’s usual artifacts are less visible.
- Most errors have moderate confidence (65–90%); only a handful exceed 95%.

5.6 Grad-CAM findings

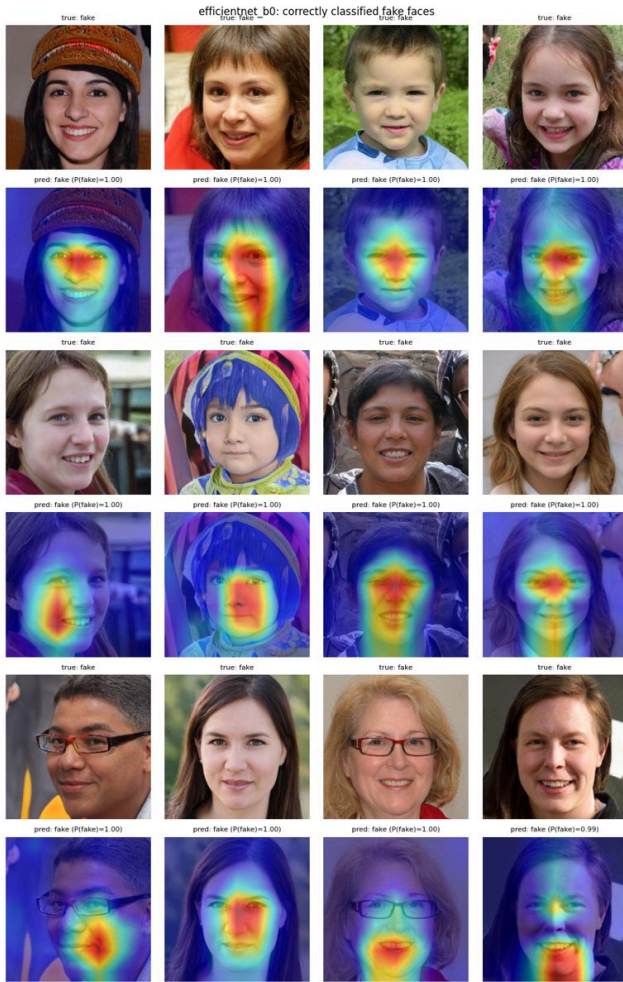


Figure 14. Correctly detected fakes: attention is tight and central (eyes, nose bridge, mouth).

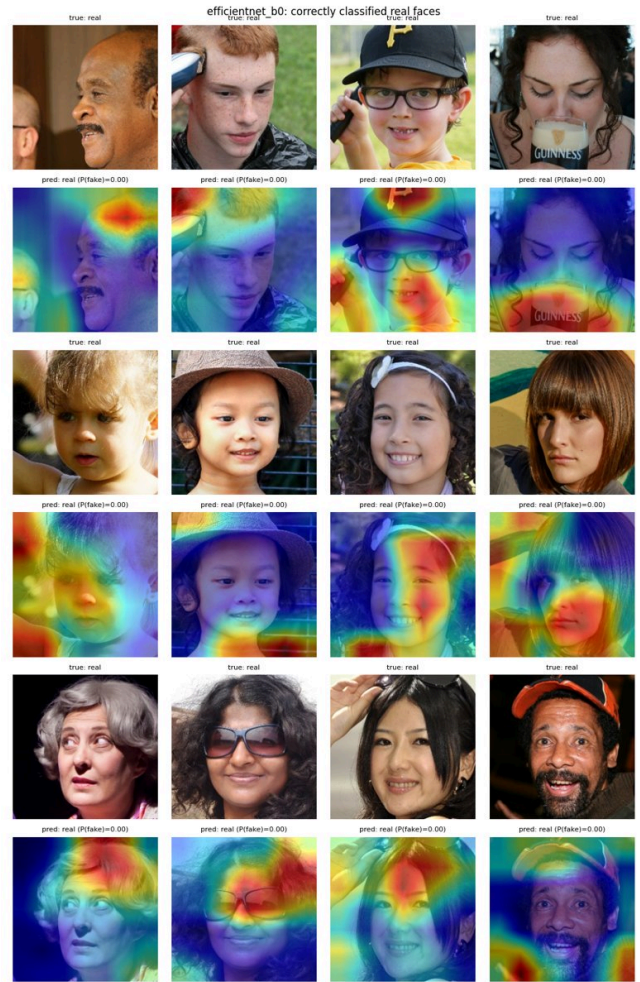


Figure 15. Correctly classified real photos: attention spread over hair, background and objects.

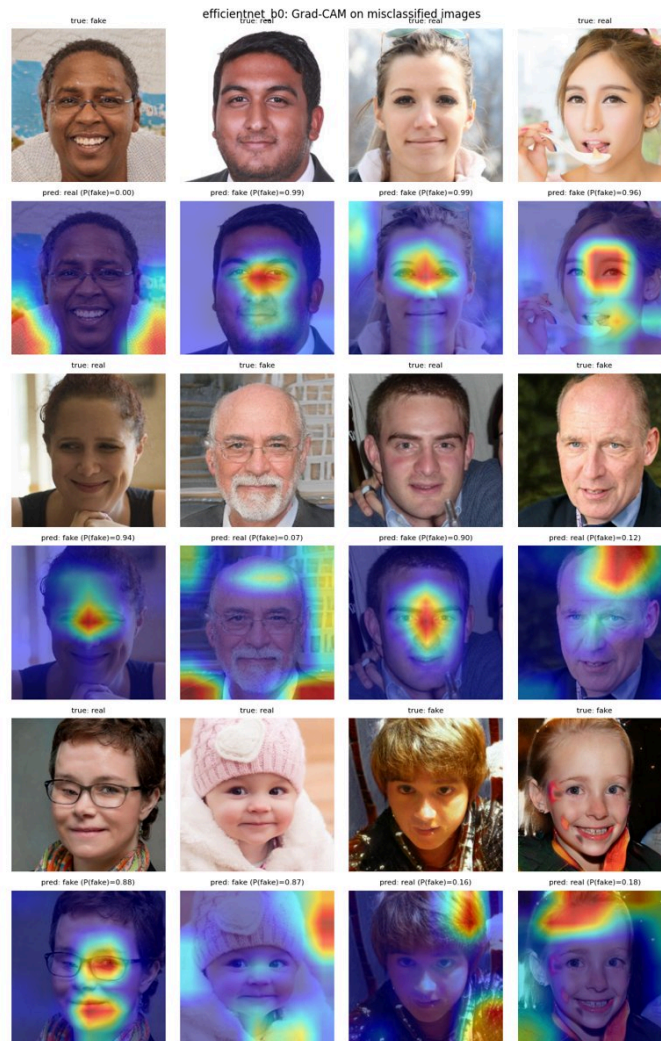


Figure 16. Grad-CAM on misclassified images (title: true label; caption: prediction).

Case	Where the model looks	Interpretation
Correct fakes	inner face: between the eyes, nose bridge, mouth/teeth	StyleGAN faces are aligned; the model checks texture/consistency where generator artifacts concentrate
Correct reals	hair, background, hats, hands, objects, borders	natural camera noise and sharp real-world context — which StyleGAN renders poorly — count as evidence for “real”
Real → fake errors	central face, like true fakes	clean, frontal, retouched portraits resemble StyleGAN output
Fake → real errors	hair, background, image edges	fakes with detailed, realistic surroundings

The heatmaps show no trivial shortcut (no fixed corner, border or watermark), which supports that the model learned meaningful cues — but these cues are tied to StyleGAN’s aligned-face style. The hand-written Grad-CAM reproduced the library maps exactly:

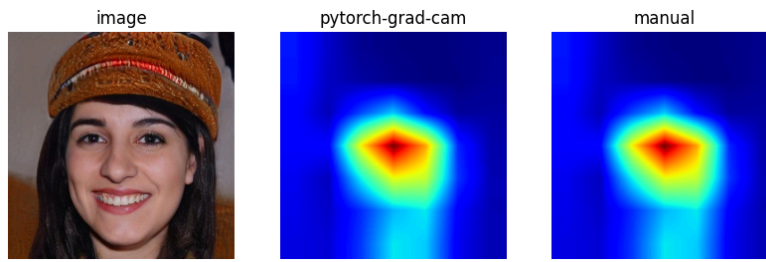


Figure 17. Input, library Grad-CAM, and our manual implementation (correlation 0.99999999).

5.7 Demo application and out-of-distribution test

The Streamlit app (`streamlit run app.py`) lets the user choose a checkpoint, upload a face image, and see the prediction (REAL/FAKE), the confidence of the predicted class, $P(fake)$ as a bar, and the Grad-CAM overlay next to the resized input. The input is preprocessed exactly like the validation set, the model is cached with `@st.cache_resource`, and one prediction plus heatmap takes about one second on a laptop CPU. The interface is deliberately plain: upload → predict → show result.

Out-of-distribution test. Three face images generated with **ChatGPT** were uploaded. All three were classified as **REAL with $\approx 100\%$ confidence** — a confident failure. This is expected and explained in Section 6: the model has only ever seen StyleGAN fakes, ChatGPT images come from an entirely different generator, they are larger and not aligned like the training faces, and a classifier always answers with one of the two classes it knows, often with high confidence.

6 Discussion

6.1 What worked

- **Transfer learning** reached near-perfect accuracy in 10 epochs (≈ 1.5 GPU-hours) instead of training from scratch.
- **The two-phase schedule** made the effect of fine-tuning visible and safe: the frozen phase set a baseline ($\approx 75\text{--}79\%$), fine-tuning added 20+ percentage points.
- **Careful protocol** — predefined splits, test set used once, checkpoint chosen on validation loss — produced numbers that agree between validation and test, so they can be trusted for this data distribution.
- **Explainability** turned a black box into an interpretable detector and explained the error patterns.
- **Engineering** — modular code, config-driven paths, resumable training, cloud notebooks — made each stage reproducible.

6.2 What did not work at first

- **Under-fitted EfficientNet (run 02)**: fine-tuning only 28% of the weights at lr $1e-5$ was too conservative. Diagnosing it from the learning curves (loss still falling, training accuracy below validation accuracy) and fixing it with a controlled second run is itself a result: the amount of fine-tuning is a key hyper-parameter.
- **Generalisation to other generators**: the ChatGPT test shows the model detects *StyleGAN*, not “fakeness” in general.

6.3 Why is the accuracy so high — is it over-fitting?

No. Over-fitting would show as a large gap between training and unseen-data performance; here training (99.72%), validation (99.88%) and test (99.84%) accuracies are within a few tenths of a percent, on 40,000 images the model never trained on. The accuracy is high because the task is narrow: every fake comes from **one** generator (StyleGAN, 2019) with a consistent fingerprint, and all faces are aligned the same way. This matches published results on the same dataset and the expectation stated in the project plan (97–99%).

6.4 Why does the model fail on ChatGPT images?

1. **Different generator, different fingerprint.** The model learned what StyleGAN artifacts look like; images from a different family of generative model do not contain them.
2. **“Real” means “not StyleGAN”.** With only two classes, anything lacking StyleGAN’s traces falls into “real”.
3. **Different input statistics.** Larger, un-aligned images resized to 224 px lose fine detail and differ from the training distribution.
4. **Over-confidence.** Neural networks are not calibrated outside their training distribution; softmax/sigmoid outputs near 1.0 do not mean certainty. This is why the app should not be used as a general-purpose deepfake detector.

7 Limitations

Limitation	Consequence
Single generator (StyleGAN) in training data	Fails on other generators, demonstrated with ChatGPT images
Aligned, cropped 256 px FFHQ-style faces only	Arbitrary photos (several faces, off-centre, large) are out of distribution; no face detector in the app
No robustness tests	Effect of JPEG re-compression, resizing, blur, screenshots or social-media processing is unknown and may erase the fingerprint
Uncalibrated confidence	High confidence on unfamiliar inputs; no “unknown” option
Coarse explanations	7×7 Grad-CAM shows roughly where, not which artifact
Still images only	No temporal cues for video deepfakes; face swaps are a different problem
Small, informal OOD test	Only three ChatGPT images were tried; a systematic OOD benchmark was not run

8 Future Work and Extensions

1. **Multi-generator training.** Add fakes from StyleGAN2/3 and diffusion models (Stable Diffusion, Midjourney, DALL·E, ChatGPT images — several such face datasets exist on Kaggle) and fine-tune on the combined data; evaluate with a **leave-one-generator-out** protocol to measure generalisation honestly.
2. **Generalisable features.** Replace or complement the CNN with frozen CLIP/ViT features and a linear probe (Ojha et al., 2023), which generalise better across generators.
3. **Frequency-domain branch.** A two-stream model (RGB + Fourier spectrum) to exploit up-sampling artifacts explicitly.
4. **Robustness augmentation.** Train with JPEG compression, blur, resizing and noise so the detector survives social-media processing.
5. **Face detection and alignment in the app** (e.g. MTCNN/RetinaFace) so real-world photos match the training format.
6. **Calibration and abstention.** Temperature scaling for trustworthy confidence, plus an “uncertain / out-of-distribution” output based on confidence or feature-space distance.
7. **Better explanations.** Grad-CAM++, Score-CAM or Integrated Gradients, and Grad-CAM at earlier layers for finer maps.
8. **Video deepfakes.** Extend to FaceForensics++/DFDC with frame-level CNN features and temporal models.
9. **Deployment at scale.** Export to ONNX/TorchScript, serve behind a REST API with GPU batching, monitor data drift and retrain as new generators appear.

9 Conclusion

We built a complete, reproducible deepfake-detection pipeline — from a verified dataset through two-phase transfer learning, rigorous evaluation and Grad-CAM explanations to a working demo. The final EfficientNet-B0 detects StyleGAN-generated faces with **99.84% accuracy and ROC-AUC 1.0000** on 20,000 unseen test images, outperforming ResNet-18 with a third of the parameters once a comparable share of the network is fine-tuned. Grad-CAM shows that it inspects the inner face for generator artifacts and uses natural context as evidence for real photos. The out-of-distribution test makes the central lesson of the project concrete: near-perfect accuracy on one generator does not imply a general deepfake detector. Extending the training data to many generators, adding robustness and calibration, and using more generalisable features are the natural next steps.

References

1. Goodfellow, I., et al. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems (NeurIPS)*.
2. Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. *CVPR*.
3. Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *ICCV*.
4. Wang, S.-Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-generated images are surprisingly easy to spot... for now. *CVPR*.
5. Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection. *Information Fusion*, 64.
6. Frank, J., Eisenhofer, T., Schönherr, L., Fischer, A., Kolossa, D., & Holz, T. (2020). Leveraging Frequency Analysis for Deep Fake Image Recognition. *ICML*.
7. Durall, R., Keuper, M., & Keuper, J. (2020). Watch your Up-Convolution: CNN Based Generative Deep Neural Networks are Failing to Reproduce Spectral Distributions. *CVPR*.
8. Ojha, U., Li, Y., & Lee, Y. J. (2023). Towards Universal Fake Image Detectors that Generalize Across Generative Models. *CVPR*.
9. Corvi, R., Cozzolino, D., Zingarini, G., Poggi, G., Nagano, K., & Verdoliva, L. (2023). On the Detection of Synthetic Images Generated by Diffusion Models. *ICASSP*.
10. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
11. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*.
12. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*.
13. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
14. xhlulu (2020). 140k Real and Fake Faces [Dataset]. Kaggle. kaggle.com/datasets/xhlulu/140k-real-and-fake-faces

Appendix A — Reproducing the results

```
# setup
git clone https://github.com/Mohd2Shaan/Deepfake-Detection-Project.git
cd Deepfake-Detection-Project
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt

# data and EDA
python scripts/download_dataset.py
python -m src.eda

# training (run 02 ResNet-18, run 02b EfficientNet-B0)
python -m src.train --model resnet18
python -m src.train --model efficientnet_b0 --unfreeze-blocks 4 --lr-finetune 1e-4

# evaluation, Grad-CAM, demo (http://localhost:8501)
python -m src.evaluate
python -m src.gradcam --checkpoint checkpoints/efficientnet_best.pth
streamlit run app.py --server.headless true
```

On Kaggle, import the notebooks in order (01 EDA → 02 training → 02b EfficientNet 4 blocks → 03 evaluation → 04 Grad-CAM), attach the dataset (and, for 02b/03/04, the previous notebook's output) and use *Save & Run All*. Each notebook's first cell sets up the repository and finds the data automatically.

Path	Content
src/	config.py, dataset.py, eda.py, model.py, train.py, evaluate.py, gradcam.py
notebooks/	executed notebooks 01, 02, 02b, 03, 04 with outputs and observations
docs/	theory and implementation note for every module (00–08) and result figures
outputs/	training histories (JSON) and generated plots
app.py	Streamlit demo

Appendix B — Anticipated viva questions

Question	Short answer
Why EfficientNet over other architectures?	Best accuracy per parameter thanks to compound scaling and MBConv+SE blocks; 4 M parameters, ≈ 0.39 GFLOPs — and empirically 6× fewer test errors than ResNet-18 here.
Why transfer learning?	Pretrained low-level features transfer to any images; converges in a few epochs with less data/compute and less over-fitting.
Why freeze first, then unfreeze?	A random head produces large gradients that would damage pretrained features; warm it up, then fine-tune the last blocks.
Why did the first EfficientNet run score only 93.7%?	Under-fitting: only 28% of weights fine-tuned at lr 1e-5; 4 blocks (92%) at lr 1e-4 gave 99.88%.
Is 99.8% over-fitting?	No — train, validation and test accuracy agree within 0.2% on 40k unseen images; the task (one generator) is easy.
Why BCEWithLogitsLoss?	Binary task with a single logit; fuses sigmoid and BCE in a numerically stable way.
How did you avoid data leakage?	Predefined splits, no reshuffling, MD5 duplicate check (0 found), test set used only once.
What does ROC-AUC = 1.0 mean?	Fakes are almost always scored higher than real images, at every threshold.
How does Grad-CAM work?	Average the gradients of the class score over each feature map of the last conv layer, use them as weights, sum the maps, ReLU, upsample.
Would this work on diffusion fakes?	No — our ChatGPT test was 3/3 wrong; it needs training data from those generators or more general features.
Computational cost?	≈ 7 min per epoch on a T4; ≈ 1 s per image including Grad-CAM on a laptop CPU.
How would you deploy at scale?	ONNX/TorchScript behind a REST API with GPU batching, face detection first, drift monitoring and periodic retraining on new generators.